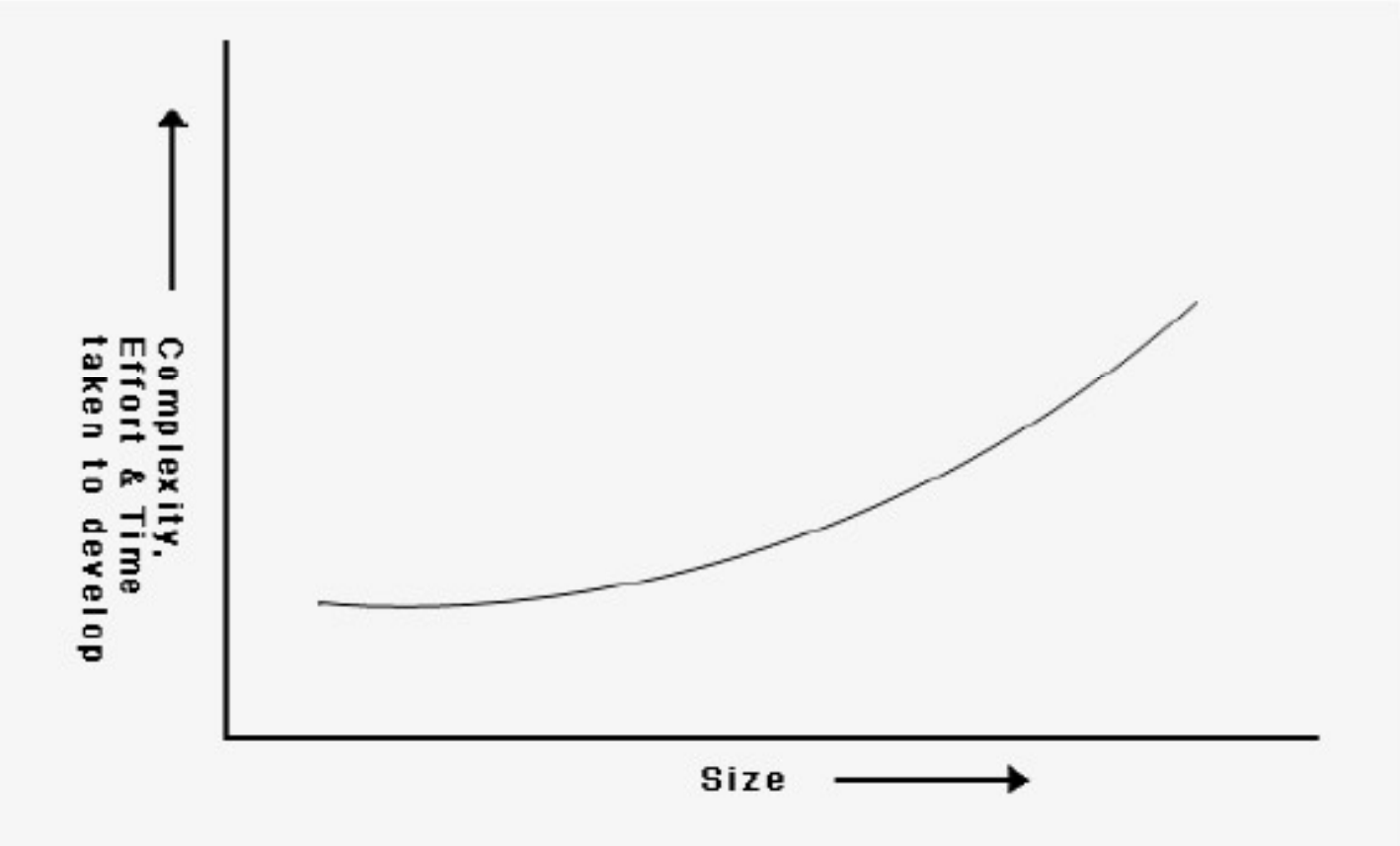
Software Engineering

S.S. Satapathy
Dept. of CSE
TU

Definition

- Software engineering can be defined as a systematic approach to develop software in a cost effective way.

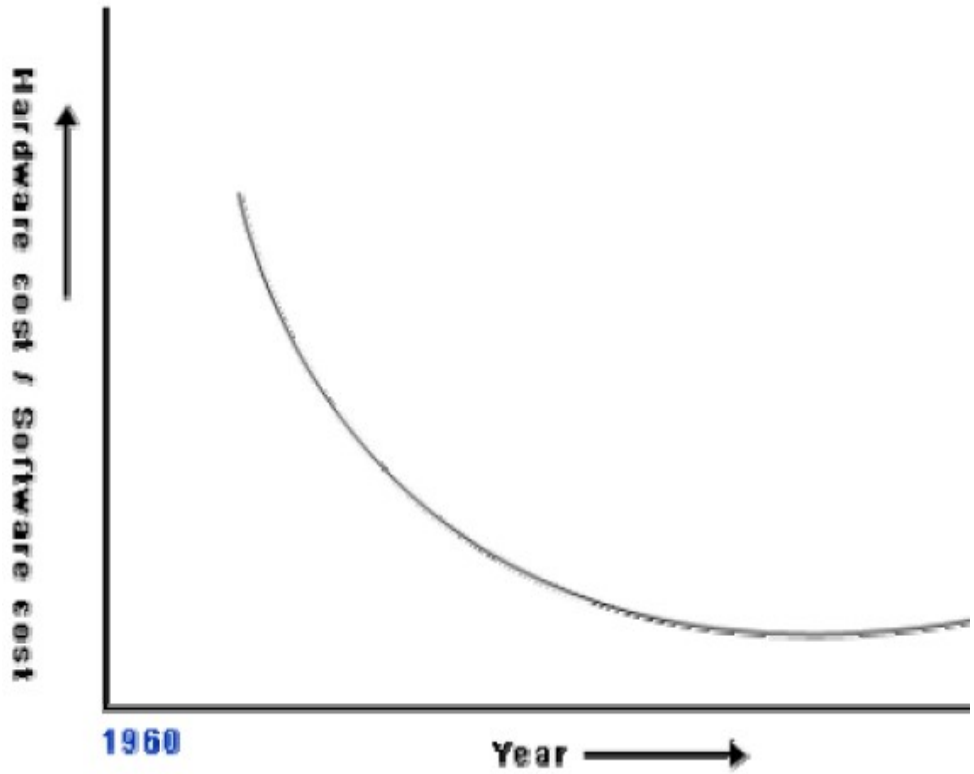
Relationship between software size and effort required is exponential



Software engineering helps to reduce the programming complexity.

Causes of & Solutions for Software Crisis

- Software crisis: Over the years, expenses on software purchases is huge compared to those on hardware purchases.
- Huge software cost is due to ineffective development
- Possible factors can be
  - larger problem sizes
  - lack of adequate training in software engineering
  - increasing skill shortage
  - low productivity



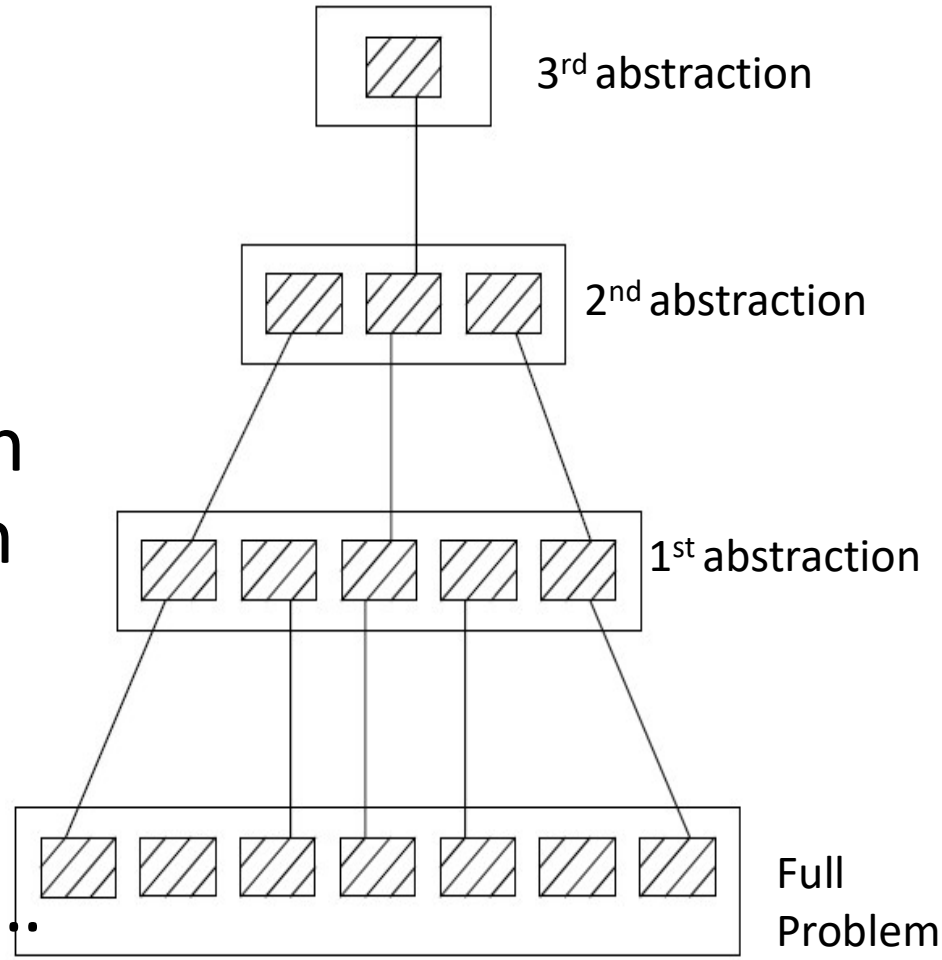
solution to the software crisis can be by using wide spread software engineering practices

Software engineering principles use two techniques to reduce problem complexity

1. Abstraction
2. Decomposition

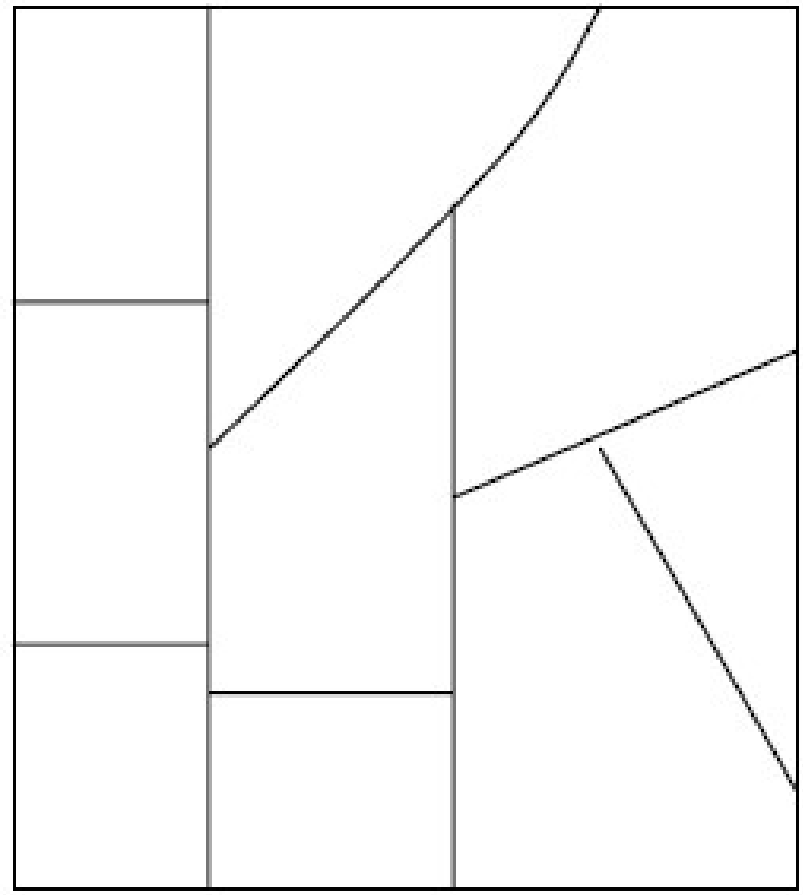
Abstraction

- Simplify problem by omitting irrelevant details
- Once the simpler problem is solved, then the omitted details can be taken into consideration to solve the next lower level abstraction, and so on...



Decomposition

- Divide a complex problem into several smaller independently problems and then solve the smaller problems are one by one
- A good decomposition of a problem should minimize interactions among various components.



Evolution of software design techniques

1. Exploratory Programming
2. Structured programming
3. Data structure-oriented design
4. Data flow-oriented design
5. Object-oriented design

Exploratory Programming (1950s)

- Programs were written in assembly language
- Programs were very small, about few hundreds of lines of assembly code
- There was no systematic approach of writing programs, programmer were writing programs as per their own individual style

Structured programming (1960s)

- High-level programming languages like FORTRAN, ALGOL, and COBOL etc came up for programming
- Programmers started giving emphasis on program's control flow structure
- Basis of the structured programming methodology was established
  - Only three programming constructs – sequence, selection, and iteration – are sufficient to express any programming logic.
  - Avoid GOTO statement
- Soon Languages like Pascal, C came up that facilitated writing programs with good control structure, that let to give emphasis on efficient data structure

Data structure-oriented design (1970s)

- Designing efficient data structure was considered important for a good program
- That helped developing efficient programs in terms of optimum time and space requirement.

Data flow-oriented design (1970s)

- Principle behind this design is that, to have a good program structure, one has to study how the data flows from input to the output of the program.
- Every program reads data and then processes that data to produce some output. Once the data flow structure is understood, it will be easy to derive the program structure.

Object-oriented design (1980s)

- This is the most recent and commonly used technique.
- It is based on a design approach in which natural objects (For example, in case of an organization, objects such as employees, pay-roll register, etc.) occurring in a problem are first identified. Each object essentially acts as a data hiding entity
- Keeping relationship (such as composition, reference and inheritance) among objects in view program structures are determined.

Exploratory style vs. modern style of software development

Sl.	Exploratory/ Earlier style	Modern style
1	Software development process purely depends on the developer	Development of a software product is very much systematic and disciplined
	based on error correction	based on error prevention
2	coding was considered synonymous with software development	coding is regarded as only a small part of the overall software development activities several development activities such as design and testing which typically require much more effort than coding
3	Software are poorly designed	Software development process is properly documented.

Several models of software development processes were/are used in software industry

Job of Modern S/W developer is difficult

- Dealing with the users
  - Ill defined requirements in many cases
  - Concerns with ease-of-use and response time
- Dealing with the technical people
  - Concerns with the code, database design, file/data structures
- Dealing with management
  - Concerns with the returns on the investment
  - Cost benefit analysis
  - Schedule

Apply Engineering Approach for

- Attempt to estimate cost/effort
- Plan and schedule work
- Involve users in defining requirements
- Identify stages in development
- Define clear milestones so that progress can be measured
- Schedule reviews both for control and quality
- Define deliverables
- Plan for extensive testing

Models of Software Development

Software Development Life Cycle

- A software life cycle model: A descriptive and diagrammatic representation of the software life cycle.
- Presents all the activities required to make a software product starting from the beginning to the release.
- It also presents the order in which these activities are to be undertaken.
- Activities can be are grouped into several phases/steps.
- Activities defined in the phases have to be executed in some specified order.

Software Development Life Cycle Contd…

- Each step has well defined objectives
- Require people with specific skills
- Takes specific inputs and produce well defined outputs
- Step defines when it may begin (entry criteria) and when to exit (Exit criteria)
- Use specific techniques, tools, guidelines and conventions

Software Development Life Cycle Contd…

- Each step
  - must be executed as per the plan, that defines durations, efforts, resources, constrains etc…
  - It must produce some information so that the management can take some corrective measures
    - Adding more resources, extend deadline etc…
  - A step end in a review (Verification and Validation)
    - Verification: Check consistence of the out put with respect to the input of the step
    - Validation: Check consistency with respect to the users requirement

Software Development Life Cycle Contd…

- A good S/W process/model
  - Facilitates early detection and removal of defects, because
    - Defects increase project cost
    - Late detection leads to higher development cost
  - It should also facilitate monitoring and improvement
    - Based on feedback
    - Permit use of new tools and techniques
    - Permit measurements

Different S/W Development Life Cycle Models

S/W Development Life Cycle Models

- The basic development activities can be mapped into different phases in several ways.
  - **Accordingly there are different life cycle models**
    1. **Classical waterfall model**
    2. **Iterative waterfall model**
    3. **Prototyping model**
    4. **Evolutionary model**
    5. **Spiral model**
- Apart from these models now a days several recent models are also in use in s/w industry

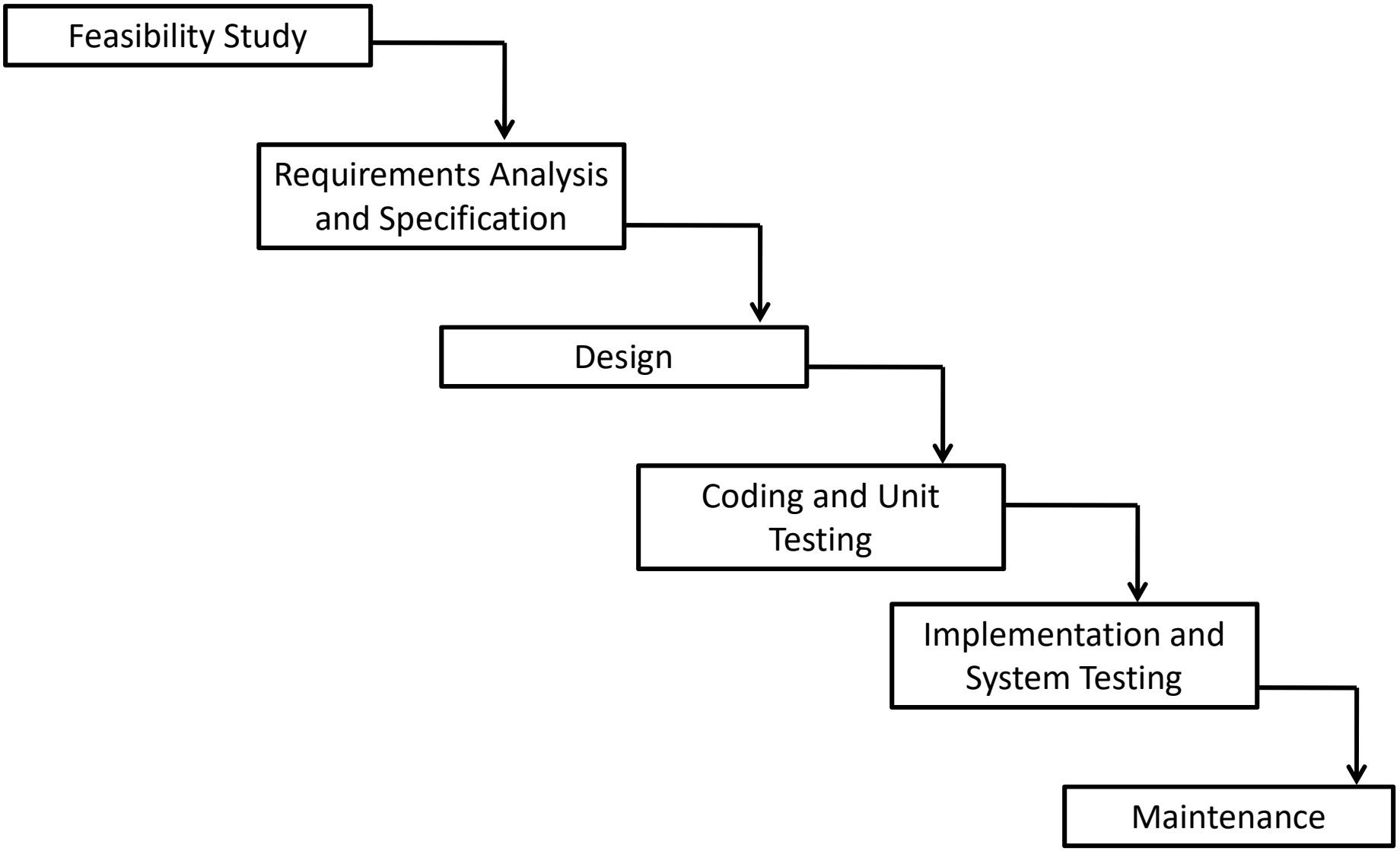
Classical Waterfall Model

- It is the most obvious way to develop software
- Though this model extensively describe the s/w development process,
  - it is not a practical model, not used as is in actual software development projects
  - considered to be a *theoretical way of developing software*
- However this model is an ideal model and all models are derived from this model

Classical Waterfall Model

- Classical waterfall model divides the life cycle into the following phases/steps:
  1. **Feasibility Study**
  2. **Requirements Analysis and Specification**
  3. **Design**
  4. **Coding and Unit Testing**
  5. **Integration and System Testing**
  6. **Maintenance**

Classical Waterfall Model contd…



Problem Definition

- What is the problem? Who think the problem exists in the current system?
- Discuss with the management/used to confirm that the problem actually exists.
- If the problem exists that need to be solved, then
  - We have a project
  - If the project is agreed, it can be assumed that the organization is going to allot fund

Problem Definition contd…

- This is the first step which is very short, may be one or two days long
- It is to understand the problem and characterize it
  - Find out cause of the problem. Few possible examples:
    - Existing system is slow, has poor response time
    - Unable to handle workload
    - Existing system is costly
    - Less accuracy/reliability
    - Output produces are not sufficient
    - Security is poor

Problem Definition contd…

- This is the first step which is very short, may be one or two days long
- It is to understand the problem and characterize it
  - Find out cause of the problem. Few possible examples:
    - Existing system is slow, has poor response time
    - Unable to handle workload
    - Existing system is costly
    - Less accuracy/reliability
    - Output produces are not sufficient
    - Security is poor

Problem Definition document

- Prepare a brief statement of the problem
  - Avoid misunderstandings
  - Get consent from the user/management
  - Keep the document very much precise, may be 1 or 2 pages
- Estimate roughly the overall cost of the project and make a tentative schedule that will be more refined in the next feasibility step

Feasibility study

- The main aim is to determine whether it would be financially and technically feasible to develop the product. Also explore operational feasibility in the organization. Activities in this phase are:
  - managers/team leads visit the client side to have a rough understanding of what is required to be done
  - Study input data to the system and output data to be produced by the system
  - Study what kind of processing is needed to be done on these data
  - Also understand the various constraints on the behavior of the system

Feasibility study contd…

- After having overall understanding of the problem, different possible solutions are explored and compare them in terms of:
  - what kind of resources required
  - what would be the cost of development
  - and what would be the development time for each solution
- Based on this analysis the best solution is considered to determine
  - whether the solution is feasible financially and technically.
  - check whether the customer budget would meet the cost of the project
  - whether they have sufficient technical expertise in the area of development

Feasibility Study report

- 1.0: Introduction
  - A brief description on the project, the environment in which the product is to be implemented, constraints associated
- 2.0: Management summary and recommendations
  - important findings about the project and suggested recommendations
- 3.0: Alternatives:
  - A presentation of alternative system specifications and criteria those are considered for final selection of the project

Feasibility Study report contd…

- 4.0: System description
  - A brief description on the system specification is the be provided
- 5.0: Cost benefit analysis
- 6.0: Legal issues if any

Requirements Analysis and Specification

- A challenging task
  - User may not know what is actually to be done and how they are going to improve the system
  - Users change their mind over time, what they confirm today may want to change tomorrow
  - They might have conflict demands
  - Analysts may not have the domain knowledge
  - Client of the s/w and actual user might not be same

Requirements Analysis and Specification contd…

- The aim of this phase is to thoroughly understand the customer requirements and prepare a SRS (Software Requirement specification) document. Activities are:
  - Requirements gathering and analysis
    - With help of through interviews and discussions with customer, collect all relevant information about the product
    - Remove contradictions, ambiguities, incompleteness and inconsistencies in the requirement
    - Once requirements are clearly understood, the next activity starts

Requirements Analysis and Specification Contd…

- Requirements specification
  - User requirements are systematically organized into a Software Requirements Specification (SRS) document
  - SRS document:
    - Important components of this document are functional requirements, the nonfunctional requirements, and the goals of implementation

Design

- The aim of this phase is to convert the requirements specified in the SRS document into some suitable structure so that they can be implemented using available programming language.
- In brief, design phase is to derive software architecture from the SRS document.

Coding and Unit Testing

- The aim of this phase is to convert the software design into source code.
  - Each component of the design is implemented as a program module.
  - The end-product of this phase is a set of program modules that have been individually tested to ensure correct working of all the individual modules.

Integration and System Testing

- In this phase individual modules developed in the coding phase are systematically combined in phase wise to develop complete software.
- Finally, when all the modules have been successfully integrated and tested, system testing is carried out to ensure that all the requirements as per the SRS document are fulfilled.
  - There are three different categories of system testing:
    - α-testing
    - B-testing
    - acceptance testing

Integration and System Testing

- α-testing: performance of software is tested by the development team
- β-testing: performance of software is tested by a friendly set of customers
- acceptance testing: performance of software is tested by the actual customer after the product delivery

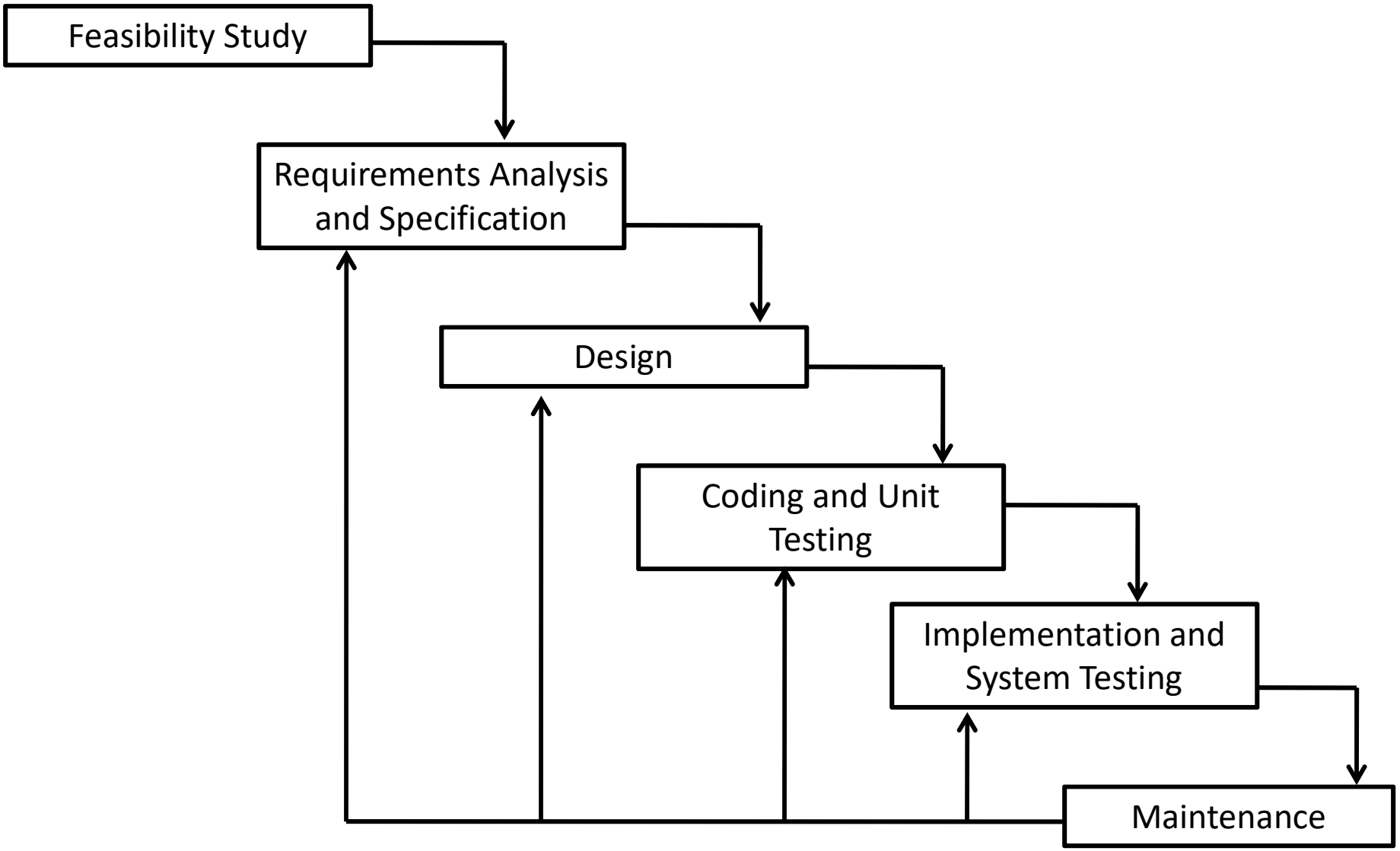
Maintenance

- Maintenance phase takes maximum effort in the development life cycle. Activities can be :
  - Correcting errors
  - Improving the implementation
  - Porting the software to work in a new environment

Shortcomings of Waterfall Model

- WF model assumes development process is linear. All the requirements are clearly known at the analysis phase and according s/w can be developed, which is often not true in all cases as all the requirements can not be predicted precisely.
  - Ex. Knowledge management system
- Requirement changes during s/w development as development process takes long time
- Therefore it might be better to develop in small increments…
- WF is considered to be document heavy:
  - All the document produced may not be useful for all type of s/w

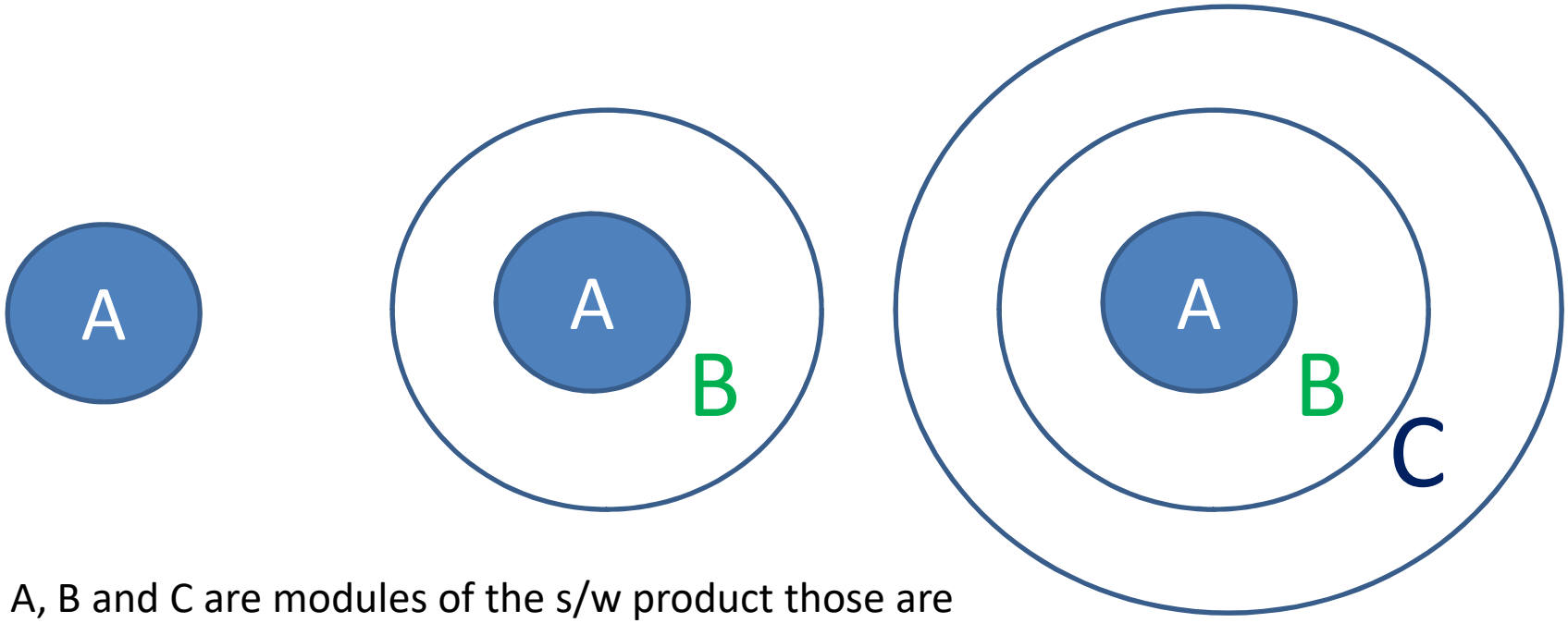
Iterative Waterfall Model



Iterative/Evolutionary/Incremental Model

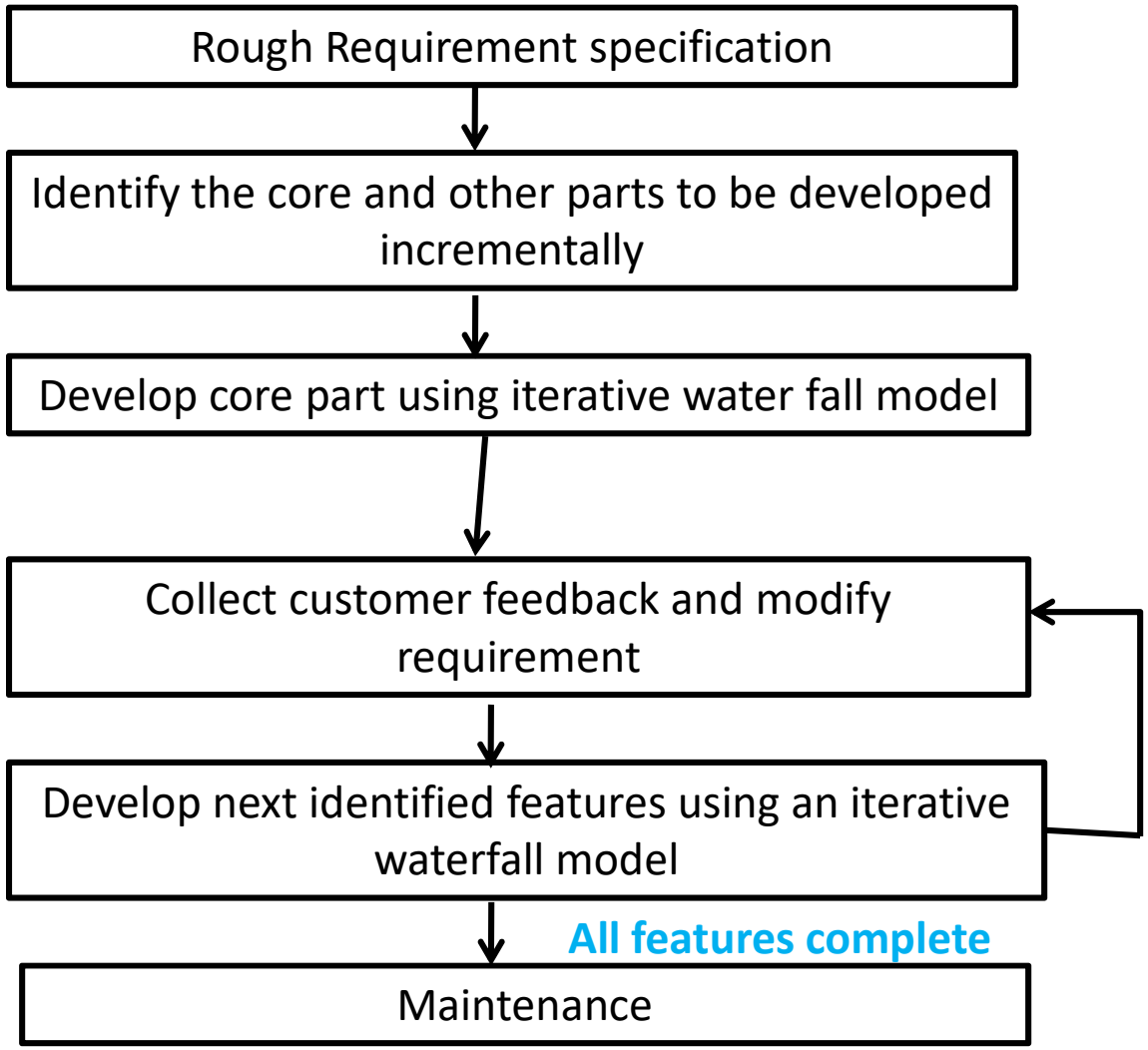
- This model is useful for products which can be used by many customers.
- Software is developed in several versions
- Here we define the initial scope and try to build a product that will fulfill some customers requirements and release the product in market
  - Any method can be used for development
- This will help to capture the market
- Then after getting customer feedback we will add more features and release in the next version

Iterative/Evolutionary/Incremental Model contd…



A, B and C are modules of the s/w product those are developed and delivered/released incrementally

Iterative/Evolutionary/Incremental Model contd…



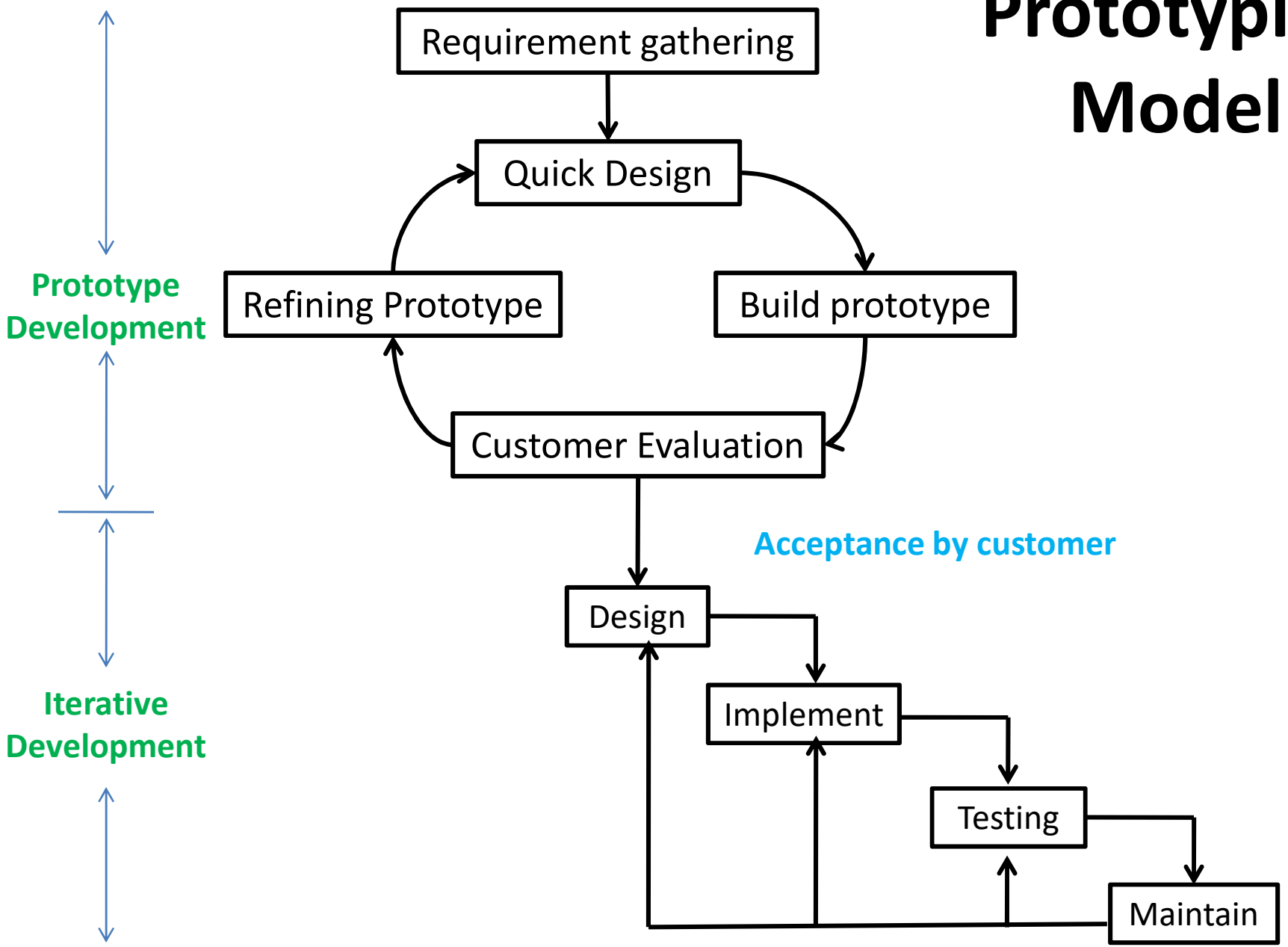
Need for a prototype in software development

- When the customer or the developer is not sure of about the requirements
  - What will the i/o
  - What will be the human/machine interface
  - What will be response time etc…
- A primitive s/w is developed with whatever information about the actual s/w to be developed is know
- It may be a working prototype or even on pen and paper
- Final product follows usual define-design-build-test phases

Prototype

- A prototype is a limited implementation of the actual software . It usually have
  - few functional capabilities
  - low reliability
  - inefficient performance compared to the actual software.
  - A prototype is usually a very crude version of the actual system.

Prototyping Model



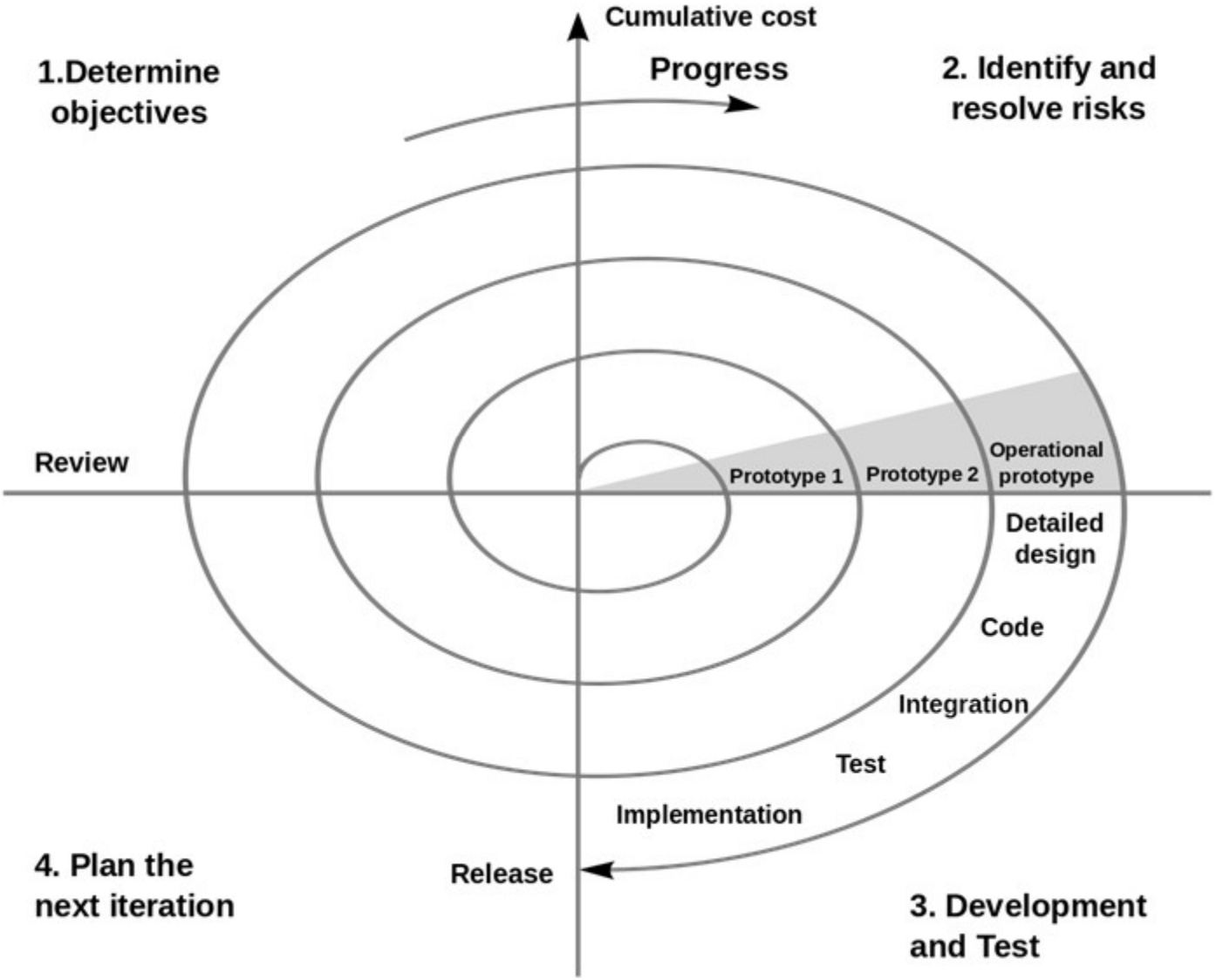
Prototype model limitations

- Customer may want the prototype itself and decide not to develop s/w any more
- Developer may continue with the approach used in developing the prototype for the real s/w development
  - that may not be that efficient
- Good tools need to be acquired for quick development of the prototyping
  - that will increase project cost

Spiral Model

- The diagrammatic view of the model looks like a spiral with many loops.
  - There is no upper limit on number of loops
  - Each loop is called as a phase
  - At the end of each phase, there will be one release of the s/w
- Each phase consists of four quadrants
- This is a risk driven s/w development

Spiral Model



Spiral Model Contd…

- Determine Objectives:
  - Identify some of the features crucial for the overall product development
  - Implementation of the identified features is the objective of the project
  - Objectives are investigated, elaborated and analyzed
- Identify and resolve the risk:
  - Based on the analysis, risks are identified and resolved by choosing one out of possible alternative solutions.

Spiral Model Contd…

- Develop next level of the product:
  - Based on the identified objectives and solution method, product is developed
  - And then product is released at the end
- Review and plan for the next phase:
  - Review product development and plan what objectives to be included in the next phase of development.

Comparison of different life-cycle models

- The classical waterfall model
  - can be considered as the basic model
  - can not be used in practical development projects
  - no mechanism to handle the errors committed during any of the phases
- The iterative waterfall model
  - probably the most widely used s/w development model
  - simple to understand and use.
  - suitable only for well-understood problems; not suitable for very large projects and for projects that are subject to many risks.

Comparison of different life-cycle models

- The prototyping model
  - suitable for projects for which either the user requirements or the underlying technical aspects are not well understood.
  - especially popular for development of the user-interface part of the projects.
- The incremental/evolutionary model
  - suitable for large problems which can be decomposed into a set of modules for development and delivery.
  - widely used for object-oriented development projects.
  - can only be used if the incremental delivery of the system is acceptable to the customer.

Comparison of different life-cycle models

- spiral model
  - also called a meta model as it encompasses all other life cycle models.
  - risk handling is inherently built into this model.
  - suitable for development of technically challenging software products that are prone to several kinds of risks.
  - not used in ordinary projects as it is much more complex than the other models